

Wireshark oefeningen Infrastructure

Dit document bevat drie Wireshark oefeningen waarmee je aan de slag kan. Wireshark is een zeer nuttig programma waarmee je netwerkverkeer kan opvangen én uitlezen.

Verwerk je antwoorden in leeg nieuw document en lever dit als PDF in op de DLO. Lever niet alles aan het eind van het vak in, maar lever elke deelopdracht zo snel mogelijk in.

Op de volgende pagina's kun je alles lezen over deze opdracht. Veel succes!

Wireshark Lab:

Getting Started & HTTP

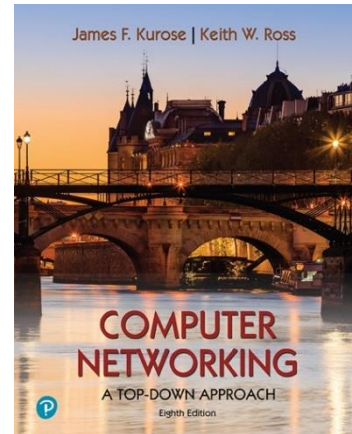
v8.1

Supplement to *Computer Networking: A Top-Down Approach*, 8th ed., J.F. Kurose and K.W. Ross

This Wireshark lab is based on the Wireshark labs Getting Started and HTTP that supplement this book.

“Tell me and I forget. Show me and I remember. Involve me and I understand.” Chinese proverb

© 2005-2021, J.F Kurose and K.W. Ross, All Rights Reserved



One's understanding of network protocols can often be greatly deepened by “seeing protocols in action” and by “playing around with protocols” – observing the sequence of messages exchanged between two protocol entities, delving down into the details of protocol operation, and causing protocols to perform certain actions and then observing these actions and their consequences. This can be done in simulated scenarios or in a “real” network environment such as the Internet. In the Wireshark labs you'll be doing in this course, you'll be running various network applications in different scenarios using your own computer. You'll observe the network protocols in your computer “in action,” interacting and exchanging messages with protocol entities executing elsewhere in the Internet. Thus, you and your computer will be an integral part of these “live” labs. You'll observe, and you'll learn, by doing.

In this first Wireshark lab, you'll get acquainted with Wireshark, and make some simple packet captures and observations.

The basic tool for observing the messages exchanged between executing protocol entities is called a **packet sniffer**. As the name suggests, a packet sniffer captures (“sniffs”) messages being sent/received from/by your computer; it will also typically store and/or display the contents of the various protocol fields in these captured messages. A packet sniffer itself is passive. It observes messages being sent and received by applications and protocols running on your computer, but never sends packets itself. Similarly, received packets are never explicitly addressed to the packet sniffer. Instead, a packet sniffer receives a *copy* of packets that are sent/received from/by application and protocols executing on your machine.

Figure 1 shows the structure of a packet sniffer. At the right of Figure 1 are the protocols (in this case, Internet protocols) and applications (such as a web browser or email client) that normally run on your computer. The packet sniffer, shown within the dashed rectangle in Figure 1 is an addition to the usual software in your computer, and consists of two parts. The **packet capture library** receives a copy of every link-layer frame that is sent from or received by your computer over a given interface (link layer, such as Ethernet or WiFi).

Recall from the discussion from section 1.5 in the text (Figure 1.24¹) that messages exchanged by higher layer protocols such as HTTP, FTP, TCP, UDP, DNS, or IP all are eventually encapsulated in link-layer frames that are transmitted over physical media such as an Ethernet cable or an 802.11 WiFi radio. Capturing all link-layer frames thus gives you all messages sent/received across the monitored link from/by all protocols and applications executing in your computer.

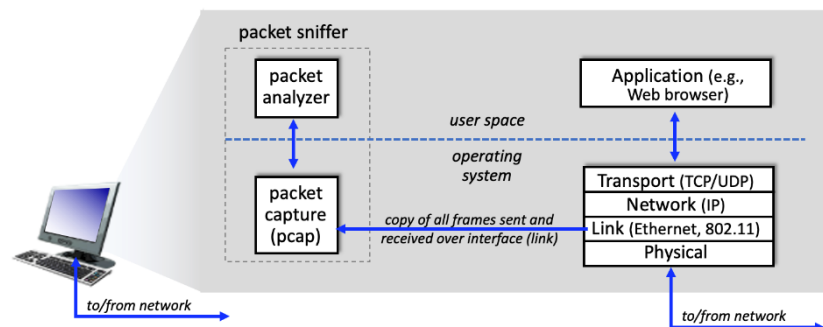


Figure 1: packet sniffer structure

The second component of a packet sniffer is the **packet analyzer**, which displays the contents of all fields within a protocol message. In order to do so, the packet analyzer must “understand” the structure of all messages exchanged by protocols. For example, suppose we are interested in displaying the various fields in messages exchanged by the HTTP protocol in Figure 1. The packet analyzer understands the format of Ethernet frames, and so can identify the IP datagram within an Ethernet frame. It also understands the IP datagram format, so that it can extract the TCP segment within the IP datagram. Finally, it understands the TCP segment structure, so it can extract the HTTP message contained in the TCP segment. Finally, it understands the HTTP protocol and so, for example, knows that the first bytes of an HTTP message will contain the string “GET,” “POST,” or “HEAD,” as shown in Figure 2.8 in the text.

We will be using the Wireshark packet sniffer [<http://www.wireshark.org/>] for these labs, allowing us to display the contents of messages being sent/received from/by protocols at different levels of the protocol stack. (Technically speaking, Wireshark is a packet analyzer that uses a packet capture library in your computer. Also, technically speaking, Wireshark captures link-layer frames as shown in Figure 1, but uses the generic term “packet” to refer to link-layer frames, network-layer datagrams, transport-layer segments, and application-layer messages, so we’ll use the less-precise “packet” term here to go along with Wireshark convention). Wireshark is a free network protocol analyzer that runs on Windows, Mac, and Linux/Unix computers. It’s an ideal packet analyzer for our labs – it is stable, has a large user base and well-documented support that includes a user-guide (http://www.wireshark.org/docs/wsug_html_chunked/), man pages (<http://www.wireshark.org/docs/man-pages/>), and a detailed FAQ (<http://www.wireshark.org/faq.html>), rich functionality that includes the capability to analyze

¹ References to figures and sections are for the 8th edition of our text, *Computer Networks, A Top-down Approach*, 8th ed., J.F. Kurose and K.W. Ross, Addison-Wesley/Pearson, 2020. Our authors’ website for this book is http://gaia.cs.umass.edu/kurose_ross. You’ll find lots of interesting open material there.

hundreds of protocols, and a well-designed user interface. It operates in computers using Ethernet, serial (PPP), 802.11 (WiFi) wireless LANs, and many other link-layer technologies.

Getting Wireshark

In order to run Wireshark, you'll need to have access to a computer that supports both Wireshark and the *libpcap* or *WinPCap* packet capture library. The *libpcap* software will be installed for you, if it is not installed within your operating system, when you install Wireshark. See <http://www.wireshark.org/download.html> for a list of supported operating systems and download sites.

Download and install the Wireshark software:

- Go to <http://www.wireshark.org/download.html> and download and install the Wireshark binary for your computer.

The Wireshark FAQ has a number of helpful hints and interesting tidbits of information, particularly if you have trouble installing or running Wireshark.

Running Wireshark

When you run the Wireshark program, you'll get a startup screen that looks something like the screen below. Different versions of Wireshark will have different startup screens – so don't panic if yours doesn't look exactly like the screen below! The Wireshark documentation states “As Wireshark runs on many different platforms with many different window managers, different styles applied and there are different versions of the underlying GUI toolkit used, your screen might look different from the provided screenshots. But as there are no real differences in functionality these screenshots should still be well understandable.” Well said.

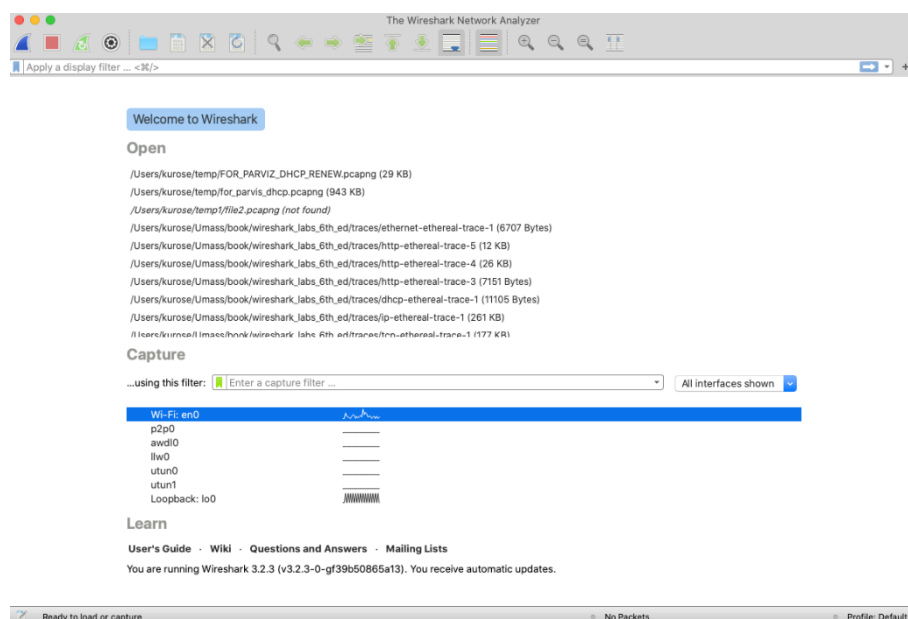


Figure 2: Initial Wireshark Screen

There's not much that's very interesting on this screen. But note that under the Capture section, there is a list of so-called interfaces. The Mac computer we're taking these

screenshots from has just one interface – “Wi-Fi en0,” (shaded in blue in Figure 2) which is the interface for Wi-Fi access. All packets to/from this computer will pass through the Wi-Fi interface, so it’s here where we’ll want to capture packets. On a Mac, double click on this interface (or on another computer locate the interface on startup page through which you are getting Internet connectivity, e.g., mostly likely a WiFi or Ethernet interface, and select that interface).

Let’s take Wireshark out for a spin! If you click on one of these interfaces to start packet capture (i.e., for Wireshark to begin capturing all packets being sent to/from that interface), a screen like the one below will be displayed, showing information about the packets being captured. Once you start packet capture, you can stop it by using the Capture pull down menu and selecting Stop (or by clicking on the red square button next to the Wireshark fin in Figure 2).²

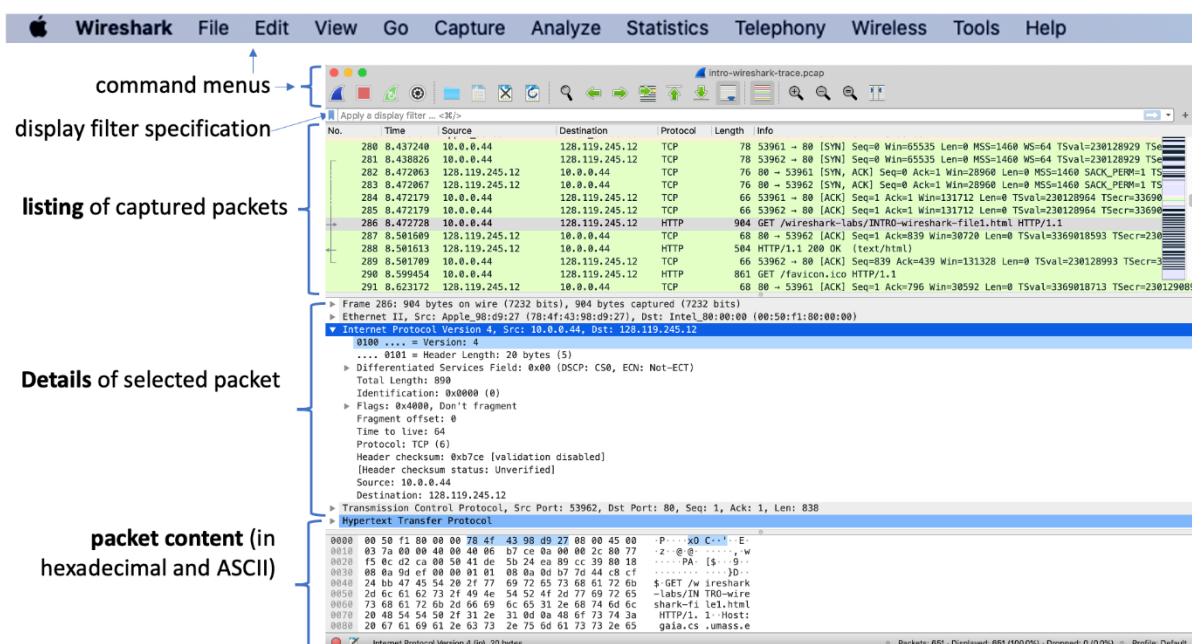


Figure 3: Wireshark window, during and after capture

This looks more interesting! The Wireshark interface has five major components:

- The **command menus** are standard pulldown menus located at the top of the Wireshark window (and on a Mac at the top of the screen as well; the screenshot in Figure 3 is from a Mac). Of interest to us now are the File and Capture menus. The File menu allows you to save captured packet data or open a file containing

² If you are unable to run Wireshark, you can still look at packet traces that were captured on one of the author’s (Jim’s) computer. You can download the zip file <http://gaia.cs.umass.edu/wireshark-labs/wireshark-traces-8.1.zip> and extract the trace file intro-wireshark-trace-1.pcap. [If you are using an Learning Management System (LMS) to answer questions in this document, you may be instructed to open a different version of this introductory trace file). Once you’ve downloaded a trace file, you can load it into Wireshark and view the trace using the *File* pull down menu, choosing *Open*, and then and then selecting the *intro-wireshark-trace* trace file. The resulting display should look similar to Figures 3 and 5. (The Wireshark user interface displays just a bit differently on different operating systems, and in different versions of Wireshark).

previously captured packet data and exit the Wireshark application. The Capture menu allows you to begin packet capture.

- The **packet-listing window** displays a one-line summary for each packet captured, including the packet number (assigned by Wireshark; note that this is *not* a packet number contained in any protocol's header), the time at which the packet was captured, the packet's source and destination addresses, the protocol type, and protocol-specific information contained in the packet. The packet listing can be sorted according to any of these categories by clicking on a column name. The protocol type field lists the highest-level protocol that sent or received this packet, i.e., the protocol that is the source or ultimate sink for this packet.
- The **packet-header details window** provides details about the packet selected (highlighted) in the packet-listing window. (To select a packet in the packet-listing window, place the cursor over the packet's one-line summary in the packet-listing window and click with the left mouse button.). These details include information about the Ethernet frame (assuming the packet was sent/received over an Ethernet interface) and IP datagram that contains this packet. The amount of Ethernet and IP-layer detail displayed can be expanded or minimized by clicking on the plus/minus boxes or right/downward-pointing triangles to the left of the Ethernet frame or IP datagram line in the packet details window. If the packet has been carried over TCP or UDP, TCP or UDP details will also be displayed, which can similarly be expanded or minimized. Finally, details about the highest-level protocol that sent or received this packet are also provided.
- The **packet-contents window** displays the entire contents of the captured frame, in both ASCII and hexadecimal format.
- Towards the top of the Wireshark graphical user interface, is the **packet display filter field**, into which a protocol name or other information can be entered in order to filter the information displayed in the packet-listing window (and hence the packet-header and packet-contents windows). In the example below, we'll use the packet-display filter field to have Wireshark hide (not display) packets except those that correspond to HTTP messages.

1. The Basic HTTP GET/response interaction

Let's begin our exploration of HTTP by downloading a very simple HTML file - one that is very short, and contains no embedded objects. Do the following:

1. Start up your web browser.
2. Start up the Wireshark packet sniffer, as described in the Introductory lab (but don't yet begin packet capture). Enter "http" (just the letters, not the quotation marks, and in lower case) in the display-filter-specification window, so that only captured HTTP messages will be displayed later in the packet-listing window. (We're only interested in the HTTP protocol here, and don't want to see the clutter of all captured packets).
3. Wait a bit more than one minute (we'll see why shortly), and then begin Wireshark packet capture.
4. Enter the following to your browser
<http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file1.html>
Your browser should display the very simple, one-line HTML file.
5. Stop Wireshark packet capture.

Your Wireshark window should look similar to the window shown in Figure 1. If you're unable to run Wireshark on a live network connection, you can download a packet trace that was created when the steps above were followed.³

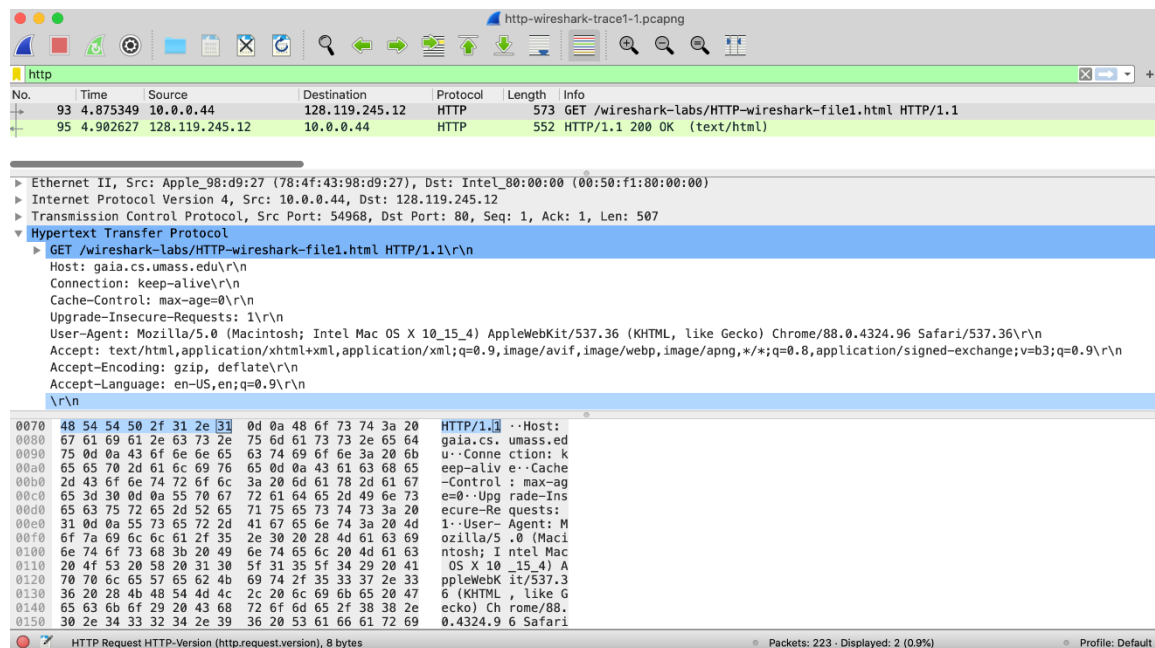


Figure 1: Wireshark Display after <http://gaia.cs.umass.edu/wireshark-labs/ HTTP-wireshark-file1.html> has been retrieved by your browser

The example in Figure 1 shows in the packet-listing window that two HTTP messages were captured: the GET message (from your browser to the gaia.cs.umass.edu web server) and the response message from the server to your browser. The packet-contents window shows details of the selected message (in this case the HTTP OK message, which is highlighted in the packet-listing window). Recall that since the HTTP message was carried inside a TCP segment, which was carried inside an IP datagram, which was carried within an Ethernet frame, Wireshark displays the Frame, Ethernet, IP, and TCP packet information as well. We want to minimize the amount of non-HTTP data displayed (we're interested in HTTP here, and will be investigating these other protocols in later labs), so make sure the boxes at the far left of the Frame, Ethernet, IP and TCP information have a plus sign or a right-pointing triangle (which means there is hidden, undisplayed information), and the HTTP line has a minus sign or a down-pointing triangle (which means that all information about the HTTP message is displayed).

(Note: You should ignore any HTTP GET and response for `favicon.ico`. If you see a reference to this file, it is your browser automatically asking the server if it (the server) has a small icon file that should

³ You can download the zip file <http://gaia.cs.umass.edu/wireshark-labs/wireshark-traces-8.1.zip> and extract the trace file `http-wireshark-trace1-1`. These trace files can be used to answer these Wireshark lab questions without actually capturing packets on your own. Each trace was made using Wireshark running on one of the author's computers, while performing the steps indicated in the Wireshark lab. Once you've downloaded a trace file, you can load it into Wireshark and view the trace using the *File* pull down menu, choosing *Open*, and then selecting the trace file name. The resulting display should look similar to Figure 1 (for the `http-wireshark-trace1-1` trace file for this HTTP lab). The Wireshark user interface displays just a bit differently on different operating systems, and in different versions of Wireshark.

be displayed next to the displayed URL in your browser. We'll ignore references to this pesky file in this lab.).

By looking at the information in the HTTP GET and response messages, answer the following questions. If you're doing this lab as part of class, your teacher will provide details about how to hand in assignments, whether written or in an LMS.⁴

1. Is your browser running HTTP version 1.0, 1.1, or 2? What version of HTTP is the server running?
2. What languages (if any) does your browser indicate that it can accept to the server?
3. What is the IP address of your computer? What is the IP address of the gaia.cs.umass.edu server?
4. What is the status code returned from the server to your browser?
5. When was the HTML file that you are retrieving last modified at the server?
6. How many bytes of content are being returned to your browser?
7. By inspecting the raw data in the packet content window, do you see any headers within the data that are not displayed in the packet-listing window? If so, name one.

In your answer to question 5 above (assuming you're running Wireshark "live", as opposed to using an earlier-recorded trace file), you might have been surprised to find that the document you just retrieved was last modified within a minute before you downloaded the document. That's because (for this particular file), the gaia.cs.umass.edu server is setting the file's last-modified time to be the current time, and is doing so once per minute. Thus, if you wait a minute between accesses, the file will appear to have been recently modified, and hence your browser will download a "new" copy of the document.

2. The HTTP CONDITIONAL GET/response interaction

Recall from Section 2.2.5 of the text, that most web browsers perform object caching and thus often perform a conditional GET when retrieving an HTTP object. Before performing the steps below, make sure your browser's cache is empty⁵. Now do the following:

- Start up your web browser, and make sure your browser's cache is cleared, as discussed above.
- Start up the Wireshark packet sniffer
- Enter the following URL into your browser
<http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file2.html>
Your browser should display a very simple five-line HTML file.
- Quickly enter the same URL into your browser again (or simply select the refresh button on your browser)
- Stop Wireshark packet capture, and enter "http" (again, in lower case without the quotation marks) in the display-filter-specification window, so that only captured HTTP messages will be displayed later in the packet-listing window.

⁴ For the author's class, when answering the following questions with hand-in assignments, students print out the GET and response messages (see the introductory Wireshark lab for an explanation of how to do this) and indicate where in the message they've found the information that answers a question. They do this by marking paper copies with a pen or annotating electronic copies with text in a colored font. There are LMS modules for teachers that allow students to answer these questions online and have answers auto-graded for these Wireshark labs at http://gaia.cs.umass.edu/kurose_ross/lms.htm

⁵ See <https://www.howtogeek.com/304218/how-to-clear-your-history-in-any-browser/> for instructions on clearing your browser cache.

If you're unable to run Wireshark on a live network connection (or unable to get your browser to issue an If-Modified-Since field on the second HTTP GET request), you can download a packet trace that was created when the steps above were followed.⁶ Answer the following questions:

8. Inspect the contents of the first HTTP GET request from your browser to the server. Do you see an "IF-MODIFIED-SINCE" line in the HTTP GET?
9. Inspect the contents of the server response. Did the server explicitly return the contents of the file? How can you tell?
10. Now inspect the contents of the second HTTP GET request from your browser to the server. Do you see an "IF-MODIFIED-SINCE:" line in the HTTP GET⁷? If so, what information follows the "IF-MODIFIED-SINCE:" header?
11. What is the HTTP status code and phrase returned from the server in response to this second HTTP GET? Did the server explicitly return the contents of the file? Explain.

3. HTML Documents with Embedded Objects

Now that we've seen how Wireshark displays the captured packet traffic for large HTML files, we can look at what happens when your browser downloads a file with embedded objects, i.e., a file that includes other objects (in the example below, image files) that are stored on another server(s).

Do the following:

- Start up your web browser, and make sure your browser's cache is cleared, as discussed above.
- Start up the Wireshark packet sniffer
- Enter the following URL into your browser
<http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file4.html>
Your browser should display a short HTML file with two images. These two images are referenced in the base HTML file. That is, the images themselves are not contained in the HTML; instead the URLs for the images are contained in the downloaded HTML file. As discussed in the textbook, your browser will have to retrieve these logos from the indicated web sites. Our publisher's logo is retrieved from the gaia.cs.umass.edu web site. The image of our 8th edition cover (one of our favorite covers) is stored at a server in France.
- Stop Wireshark packet capture, and enter "http" in the display-filter-specification window, so that only captured HTTP messages will be displayed.

⁶ If you're unable to run Wireshark on a live network connection, you can download the zip file <http://gaia.cs.umass.edu/wireshark-labs/wireshark-traces-8.1.zip> and extract the trace file http-wireshark-trace2-1.

⁷ *Hint:* ideally, you should see an If-Modified-Since header since you've just downloaded this page a few seconds ago. However, depending on the browser you're using, and the format of the server's earlier response to your initial GET, your browser may not include an If-Modified-Since even if the document has been downloaded and caches. The Chrome browser is pretty good at regularly using If-Modified-Since. But Safari and Firefox are much more finicky about when to use If-Modified-Since. Life isn't always as easy in practice as it is in theory!

Answer the following questions⁸:

12. How many HTTP GET request messages did your browser send? To which Internet addresses were these GET requests sent?
13. Can you tell whether your browser downloaded the two images serially, or whether they were downloaded from the two web sites in parallel? Explain.

4. HTTP Authentication

Finally, let's try visiting a web site that is password-protected and examine the sequence of HTTP message exchanged for such a site. The URL http://gaia.cs.umass.edu/wireshark-labs/protected_pages/HTTP-wireshark-file5.html is password protected. The username is "wireshark-students" (without the quotes), and the password is "network" (again, without the quotes). So let's access this "secure" password-protected site. Do the following:

- Make sure your browser's cache is cleared, as discussed above, and close down your browser. Then, start up your browser
- Start up the Wireshark packet sniffer
- Enter the following URL into your browser
http://gaia.cs.umass.edu/wireshark-labs/protected_pages/HTTP-wireshark-file5.html
Type the requested user name and password into the pop up box.
- Stop Wireshark packet capture, and enter "http" in the display-filter-specification window, so that only captured HTTP messages will be displayed later in the packet-listing window.
- *Note:* If you are unable to run Wireshark on a live network connection, you can use the "classic" http-ethereal-trace-5 packet trace, or other additional traces, as notes in footnote 2, to answer the questions below.

Now let's examine the Wireshark output. You might want to first read up on HTTP authentication by reviewing the easy-to-read material on "HTTP Access Authentication Framework" at [http://frontier.userland.com/stories/storyReader\\$2159](http://frontier.userland.com/stories/storyReader$2159)

Answer the following questions⁹:

14. What is the server's response (status code and phrase) in response to the initial HTTP GET message from your browser?
15. When your browser's sends the HTTP GET message for the second time, what new field is included in the HTTP GET message?

The username (wireshark-students) and password (network) that you entered are encoded in the string of characters (d2lyZXNoYXJrLXN0dWRIbnRzOm5ldHdvcms=) following the "Authorization: Basic" header in the client's HTTP GET message. While it may appear that your username and password are encrypted, they are simply encoded in a format

⁸ If you're unable to run Wireshark on a live network connection, you can download the zip file <http://gaia.cs.umass.edu/wireshark-labs/wireshark-traces-8.1.zip> and extract the trace file http-wireshark-trace4-1.

⁹ If you're unable to run Wireshark on a live network connection, you can download the zip file <http://gaia.cs.umass.edu/wireshark-labs/wireshark-traces-8.1.zip> and extract the trace file http-wireshark-trace5-1.

known as Base64 format. The username and password are *not* encrypted! To see this, go to <http://www.motobit.com/util/base64-decoder-encoder.asp> and enter the base64-encoded string d2lyZXNoYXJrLXN0dWRIbnRz and decode. *Voila!* You have translated from Base64 encoding to ASCII encoding, and thus should see your username! To view the password, enter the remainder of the string Om5ldHdvcms= and press decode. Since anyone can download a tool like Wireshark and sniff packets (not just their own) passing by their network adaptor, and anyone can translate from Base64 to ASCII (you just did it!), it should be clear to you that simple passwords on WWW sites are not secure unless additional measures are taken.

Fear not! As we will see in Chapter 8, there are ways to make WWW access more secure. However, we'll clearly need something that goes beyond the basic HTTP authentication framework!

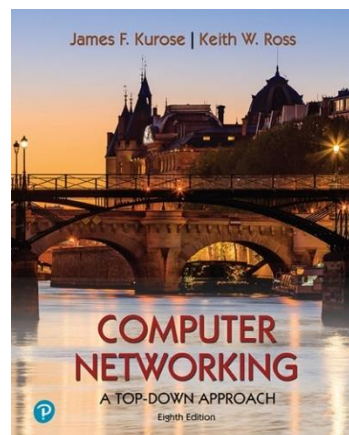
Wireshark Lab:

UDP & TCP v8.1

Supplement to *Computer Networking: A Top-Down Approach*, 8th ed., J.F. Kurose and K.W. Ross

“Tell me and I forget. Show me and I remember. Involve me and I understand.” Chinese proverb

© 2005-2021, J.F Kurose and K.W. Ross, All Rights Reserved



In this lab, we'll take a quick look at the UDP transport protocol. As we saw in Chapter 3 of the text¹⁰, UDP is a streamlined, no-frills protocol. You may want to re-read section 3.3 in the text before doing this lab. Because UDP is simple and sweet, we'll be able to cover it pretty quickly in this lab. So if you've another appointment to run off to in 30 minutes, no need to worry, as you should be able to finish this lab with ample time to spare.

At this stage, you should be a Wireshark expert. Thus, we are not going to spell out the steps as explicitly as in earlier labs. In particular, we are not going to provide example screenshots for all the steps.

1. The Assignment

Start capturing packets in Wireshark and then do something that will cause your host to send and receive several UDP packets. It's also likely that just by doing nothing (except capturing packets via Wireshark) that some UDP packets sent by others will appear in your trace. In particular, the Domain Name System (DNS) protocol (see section 2.4 in the text; and the DNS Wireshark Lab) typically sends DNS query and response messages inside of UDP, so it's likely that you'll find some DNS messages (and therefore UDP packets) in your trace.

Specifically you can try out the `nslookup` command, which invokes the underlying DNS protocol, which in turn will send UDP segments from/to the host issuing the `nslookup`. `nslookup` is available in most Microsoft, Apple IOS, and Linux operating systems. To run `nslookup` you just type the `nslookup` command on the command line in a DOS window, Mac IOS terminal window, or Linux shell. Figure 1 is a screenshot of running `nslookup` on the Linux command line on the newworld.cs.umass.edu host located in the CS Department at the University of Massachusetts (UMass) campus, to display the IP address of `www.nyu.edu`.

¹⁰ References to figures and sections are for the 8th edition of our text, *Computer Networks, A Top-down Approach*, 8th ed., J.F. Kurose and K.W. Ross, Addison-Wesley/Pearson, 2020. Our website for this book is http://gaia.cs.umass.edu/kurose_ross You'll find lots of interesting open material there.

```

newworld.cs.umass.edu> nslookup www.nyu.edu
Server:      128.119.240.1
Address:     128.119.240.1#53

Non-authoritative answer:
www.nyu.edu canonical name = WEB.GSLB.nyu.edu.
Name:   WEB.GSLB.nyu.edu
Address: 216.165.47.12
Name:   WEB.GSLB.nyu.edu
Address: 2607:f600:1002:6113::100

```

Figure 1: the basic `nslookup` command

We don't need to go into any more details about `nslookup` or DNS, as we're just interested in getting a few UDP segments into Wireshark, and we promised this lab would be short!

After starting packet capture on Wireshark, run `nslookup` for a hostname that you haven't visited for a while. Then stop packet capture, set your Wireshark packet filter so that Wireshark only displays the UDP segments sent and received at your host. Pick the first UDP segment and expand the UDP fields in the details window. If you are unable to find UDP segments in your trace or are unable to run Wireshark on a live network connection, you can download a packet trace containing some UDP segments¹¹.

Answer the following questions¹². If you're doing this lab as part of class, your teacher will provide details about how to hand in assignments, whether written or in an LMS.

1. Select the first UDP segment in your trace. How many fields there are in the UDP header? (You shouldn't look in the textbook! Answer these questions directly from what you observe in the packet trace.) What are the names of these fields?
2. By consulting the displayed information in Wireshark's packet content field for this packet (or by consulting the textbook), what is the length (in bytes) of each of the UDP header fields?
3. The value in the Length field is the length of what? (You can consult the text for this answer). Verify your claim with your captured UDP packet.
4. What is the maximum number of bytes that can be included in a UDP payload? (Hint: the answer to this question can be determined by your answer to 2. above)
5. What is the largest possible source port number? (Hint: see the hint in 4.)
6. What is the protocol number for UDP? Give your answer in decimal notation. To answer this question, you'll need to look into the Protocol field of the IP datagram containing this UDP segment (see Figure 4.13 in the text, and the discussion of IP header fields).
7. Examine the pair of UDP packets in which your host sends the first UDP packet and the second UDP packet is a reply to this first UDP packet. (Hint: for a second packet

¹¹ You can download the zip file <http://gaia.cs.umass.edu/wireshark-labs/wireshark-traces-8.1.zip> and extract the trace file `dns-wireshark-trace1-1`. This trace file can be used to answer this Wireshark lab without actually capturing packets on your own. This trace was made using Wireshark running on one of the author's computers, while performing the steps indicated in this Wireshark lab. Once you've downloaded a trace file, you can load it into Wireshark and view the trace using the *File* pull down menu, choosing *Open*, and then selecting the trace file name.

¹² For the author's class, when answering the following questions with hand-in assignments, students sometimes need to print out specific packets (see the introductory Wireshark lab for an explanation of how to do this) and indicate where in the packet they've found the information that answers a question. They do this by marking paper copies with a pen or annotating electronic copies with text in a colored font. There are also learning management system (LMS) modules for teachers that allow students to answer these questions online and have answers auto-graded for these Wireshark labs at http://gaia.cs.umass.edu/kurose_ross/lms.htm

to be sent in response to a first packet, the sender of the first packet should be the destination of the second packet). Describe the relationship between the port numbers in the two packets.

2. Capturing a bulk TCP transfer from your computer to a remote server

In this lab, we'll investigate the behavior of the celebrated TCP protocol in detail. We'll do so by analyzing a trace of the TCP segments sent and received in transferring a 150KB file (containing the text of Lewis Carrol's *Alice's Adventures in Wonderland*) from your computer to a remote server.

Before beginning this lab, you'll probably want to review sections 3.5 and 3.7 in the text¹³.

Before beginning our exploration of TCP, we'll need to use Wireshark to obtain a packet trace of the TCP transfer of a file from your computer to a remote server. You'll do so by accessing a Web page that will allow you to enter the name of a file stored on your computer (which contains the ASCII text of *Alice in Wonderland*), and then transfer the file to a Web server using the HTTP POST method (see section 2.2.3 in the text). We're using the POST method rather than the GET method as we'd like to transfer a large amount of data *from* your computer to another computer. Of course, we'll be running Wireshark during this time to obtain the trace of the TCP segments sent and received from your computer.

Do the following:

- Start up your web browser. Go the <http://gaia.cs.umass.edu/wireshark-labs/alice.txt> and retrieve an ASCII copy of *Alice in Wonderland*. Store this as a .txt file somewhere on your computer.
- Next go to <http://gaia.cs.umass.edu/wireshark-labs/TCP-wireshark-file1.html>.
- You should see a screen that looks like Figure 2.

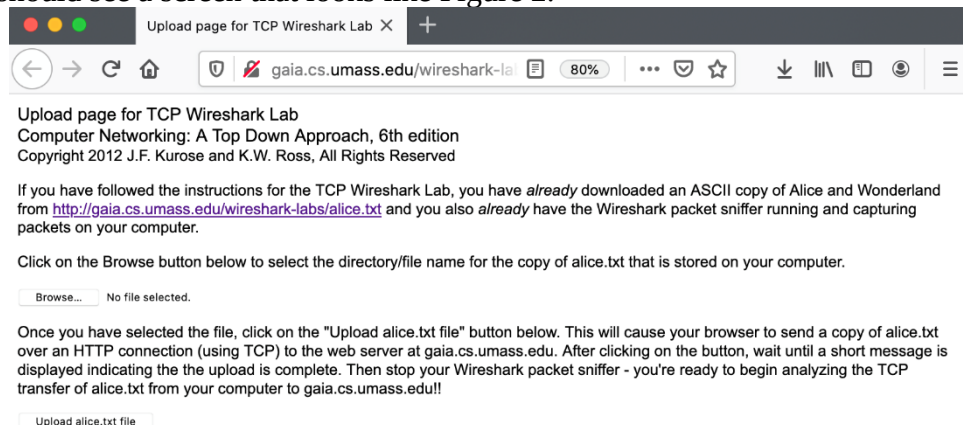


Figure 2: Page to upload the alice.txt file from your computer to gaia.cs.umass.edu

- Use the *Browse* button in this form to the file on your computer that you just created containing *Alice in Wonderland*. Don't press the "*Upload alice.txt file*" button yet.

¹³ References to figures and sections are for the 8th edition of our text, *Computer Networks, A Top-down Approach, 8th ed.*, J.F. Kurose and K.W. Ross, Addison-Wesley/Pearson, 2020. Our website for this book is http://gaia.cs.umass.edu/kurose_ross You'll find lots of interesting open material there.

- Now start up Wireshark and begin packet capture (see the earlier Wireshark labs if you need a refresher on how to do this).
- Returning to your browser, press the “*Upload alice.txt file*” button to upload the file to the `gaia.cs.umass.edu` server. Once the file has been uploaded, a short congratulations message will be displayed in your browser window.
- Stop Wireshark packet capture. Your Wireshark window should look similar to the window shown in Figure 3.

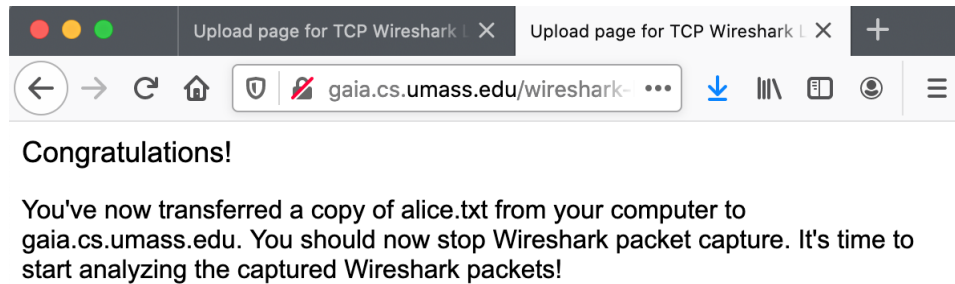


Figure 3: Success! You’ve uploaded a file to `gaia.cs.umass.edu` and have hopefully captured a Wireshark packet trace while doing so.

If you are unable to run Wireshark on a live network connection, you can download a packet trace that was captured while following the steps above on one of the author’s computers¹⁴. In addition, you may well find it valuable to download this trace even if you’ve captured your own trace and use it, as well as your own trace, when you explore the questions below.

2. A first look at the captured trace

Before analyzing the behavior of the TCP connection in detail, let’s take a high-level view of the trace.

First, filter the packets displayed in the Wireshark window by entering “tcp” (lowercase, no quotes, and don’t forget to press return after entering!) into the display filter specification window towards the top of the Wireshark window.

What you should see is series of TCP and HTTP messages between your computer and `gaia.cs.umass.edu`. You should see the initial three-way handshake containing a SYN message. You should see an HTTP POST message. In recent versions of Wireshark, you’ll see “[TCP segment of a reassembled PDU]” in the Info column of the Wireshark display to indicate that this TCP segment contained data that belonged to an upper layer protocol message (in our case here, HTTP). You should also see TCP ACK segments being returned from `gaia.cs.umass.edu` to your computer.

¹⁴ You can download the zip file <http://gaia.cs.umass.edu/wireshark-labs/wireshark-traces-8.1.zip> and extract the trace file `tcp-wireshark-trace1-1`. This trace file can be used to answer this Wireshark lab without actually capturing packets on your own. This trace was made using Wireshark running on one of the author’s computers, while performing the steps indicated in this Wireshark lab. Once you’ve downloaded a trace file, you can load it into Wireshark and view the trace using the *File* pull down menu, choosing *Open*, and then selecting the trace file name.

1. What is the IP address and TCP port number used by the client computer (source) that is transferring the `alice.txt` file to `gaia.cs.umass.edu`? To answer this question, it's probably easiest to select an HTTP message and explore the details of the TCP packet used to carry this HTTP message, using the "details of the selected packet header window" (refer to Figure 2 in the "Getting Started with Wireshark" Lab if you're uncertain about the Wireshark windows).
2. What is the IP address of `gaia.cs.umass.edu`? On what port number is it sending and receiving TCP segments for this connection?

Since this lab is about TCP rather than HTTP, now change Wireshark's "listing of captured packets" window so that it shows information about the TCP segments containing the HTTP messages, rather than about the HTTP messages. This is what we're looking for—a series of TCP segments sent between your computer and `gaia.cs.umass.edu`!

3. TCP Basics

Answer the following questions for the TCP segments:

3. What is the *sequence number* of the TCP SYN segment that is used to initiate the TCP connection between the client computer and `gaia.cs.umass.edu`? What is it in this TCP segment that identifies the segment as a SYN segment?
4. What is the *sequence number* of the SYNACK segment sent by `gaia.cs.umass.edu` to the client computer in reply to the SYN? What is it in the segment that identifies the segment as a SYNACK segment? What is the value of the Acknowledgement field in the SYNACK segment? How did `gaia.cs.umass.edu` determine that value?
5. What is the sequence number of the TCP segment containing the header of the HTTP POST command? Note that in order to find the POST message header, you'll need to dig into the packet content field at the bottom of the Wireshark window, *looking for a segment with the ASCII text "POST" within its DATA field*^{15,16}.
6. Consider the TCP segment containing the HTTP "POST" as the first segment in the data transfer part of the TCP connection.
 - At what time was the first segment (the one containing the HTTP POST) in the data-transfer part of the TCP connection sent?
 - At what time was the ACK for this first data-containing segment received?
 - What is the RTT for this first data-containing segment?
 - What is the RTT value the second data-carrying TCP segment and its ACK?
*Note: Wireshark has a nice feature that allows you to plot the RTT for each of the TCP segments sent. Select a TCP segment in the "listing of captured packets" window that is being sent from the client to the `gaia.cs.umass.edu` server. Then select: **Statistics->TCP Stream Graph->Round Trip Time Graph**.*
7. Are there any retransmitted segments in the trace file? What did you check for (in the trace) in order to answer this question?

¹⁵ Hint: this TCP segment is sent by the client soon (but not always immediately) after the SYNACK segment is received from the server.

¹⁶ Note that if you filter to only show "http" messages, you'll see that the TCP segment that Wireshark associates with the HTTP POST message is the *last* TCP segment in the connection (which contains the text at the *end* of `alice.txt`: "THE END") and *not* the first data-carrying segment in the connection. Students (and teachers!) often find this unexpected and/or confusing.

Wireshark Lab:

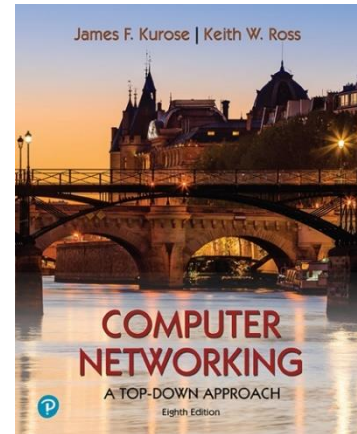
TLS v8.1

Supplement to *Computer Networking: A Top-Down Approach*, 8th ed., J.F. Kurose and K.W. Ross

This is an abbreviated version of the original Wireshark Lab for TLS.

“Tell me and I forget. Show me and I remember. Involve me and I understand.” Chinese proverb

© 2005-2021, J.F Kurose and K.W. Ross, All Rights Reserved



In this lab, we'll investigate Transport Layer Security (known as TLS) and aspects of the authentication, data integrity, and confidentiality services provided by TLS. TLS is the successor to the now-deprecated Secure Sockets Layer (SSL). So, it'd be preferable to do this v8.1 TLS Wireshark lab, rather than the v8.0 SSL Wireshark Lab.

We'll investigate TLS by analyzing a Wireshark packet trace captured during the retrieval of a web page via HTTPS - a secure version of HTTP, which implements TLS on top of HTTP. We'll look at TLS's client-server handshaking protocol in some detail, since that's where most of the interesting action happens. You may want to review Section 8.6 in the text¹⁷.

1. Capturing packets in an TLS session

The first step in this lab is to capture the packets in an TLS session. To do this, you should startup Wireshark and begin packet capture, retrieve the homepage from <https://www.cics.umass.edu> using the browser of your choice, and then stop Wireshark packet capture. The 's' after 'http' will cause the **Hypertext Transfer Protocol Secure (HTTPS)** - an extension of HTTP – to be used to securely retrieve the homepage from www.cics.umass.edu. Here, “securely” means that the www.cics.umass.edu server will be authenticated by your web browser, that the transmission of your client HTTP GET request and the server's reply will be encrypted, and the integrity of all message content will be cryptographically verified. Of course, the authentication, integrity and encryption of a computer science department's webpage may not be as critical as that for Internet commerce and banking sites, but the same TLS protocol and TLS messages are used in all cases.

If you're unable to run Wireshark on a live network connection, you can download a packet trace that was captured while following the steps above on one of the author's computers¹⁸.

¹⁷ References to figures and sections are for the 8th edition of our text, *Computer Networks, A Top-down Approach*, 8th ed., J.F. Kurose and K.W. Ross, Addison-Wesley/Pearson, 2020. Our authors' website for this book is http://gaia.cs.umass.edu/kurose_ross. You'll find lots of interesting open material there.

¹⁸ You can download the zip file <http://gaia.cs.umass.edu/wireshark-labs/wireshark-traces-8.1.zip> and extract the trace file `tls-wireshark-trace1`. This trace file can be used to answer this Wireshark lab without actually

In addition, you may well find it valuable to download this trace even if you've captured your own trace and use it, as well as your own trace, when you explore the questions below.

2. A first look at the captured trace

Let's first set Wireshark's display so that only the packets to and from `www.cics.umass.edu`, whose IP address is `128.119.240.84`, are displayed. To do this, enter

```
ip.addr == 128.119.240.84
```

in Wireshark's display filter window. Your screen should look similar to what is shown in Figure 1.

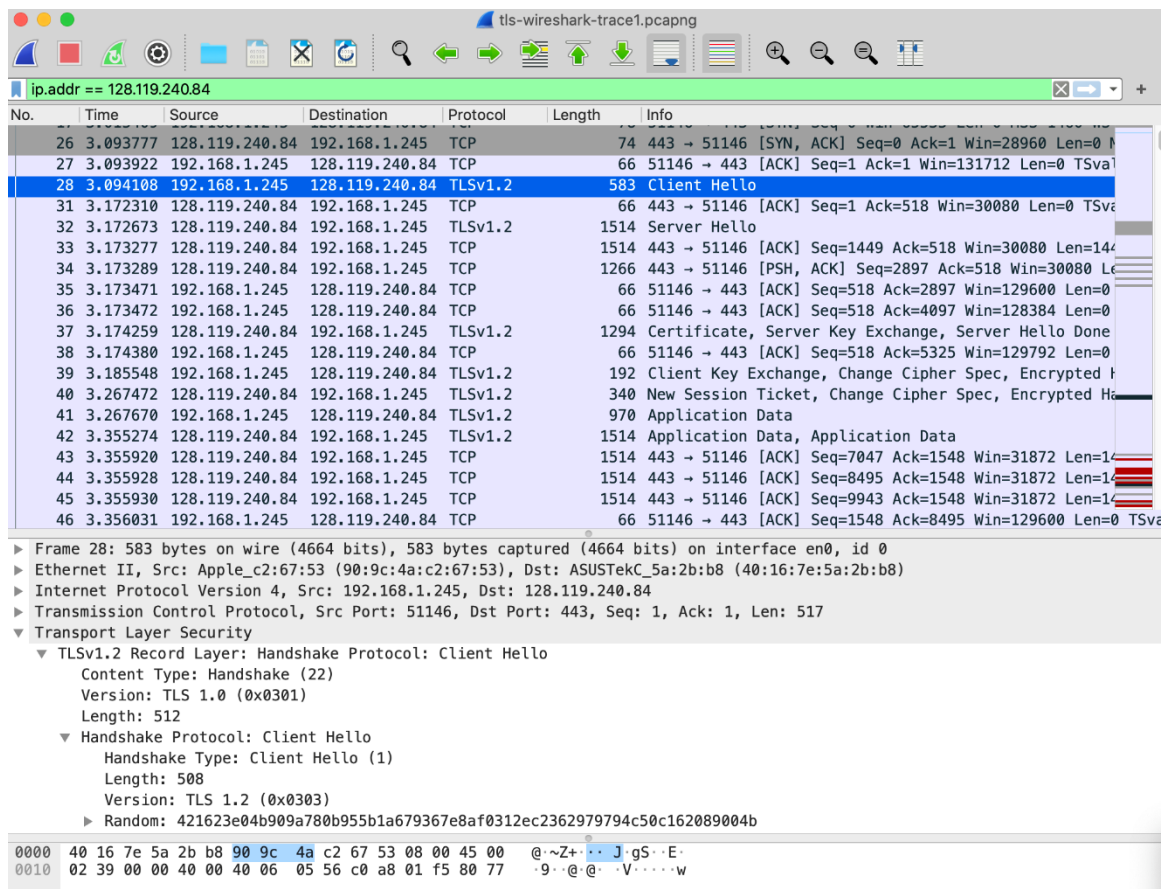


Figure 1: Wireshark display showing TCP and TLS message to/from `128.119.240.84`

It's important to keep in mind that an Ethernet frame (containing an IP datagram containing an TCP segment) may contain one or more TLS records. (This is very different from HTTP, for which each frame contains either one complete HTTP message or a portion of a HTTP message.) Also, a TLS record may not completely fit into an Ethernet frame, in which case multiple frames will be needed to carry the record.

capturing packets on your own. This trace was made using Wireshark running on one of the author's computers, while performing the steps indicated in this Wireshark lab. Once you've downloaded a trace file, you can load it into Wireshark and view the trace using the *File* pull down menu, choosing *Open*, and then selecting the trace file name.

In answering the questions below¹⁹, you can use either your own live trace, or use the Wireshark captured packet file *TLS-wireshark-trace1* in <http://gaia.cs.umass.edu/wireshark-labs/wireshark-traces-8.1.zip>

We've said earlier that HTTPS implements TLS running "over" TCP. That means that a TCP connection must first be established between your browser and the web server for www.cics.umass.edu before TLS and HTTP messages can be exchanged, just as we saw with the vanilla (non-TLS) HTTP protocol.

1. What is the packet number in your trace that contains the initial TCP SYN message? (By "packet number," we meant the number in the "No." column at the left of the Wireshark display, not the sequence number in the TCP segment itself).
2. Is the TCP connection set up before or after the first TLS message is sent from client to server?

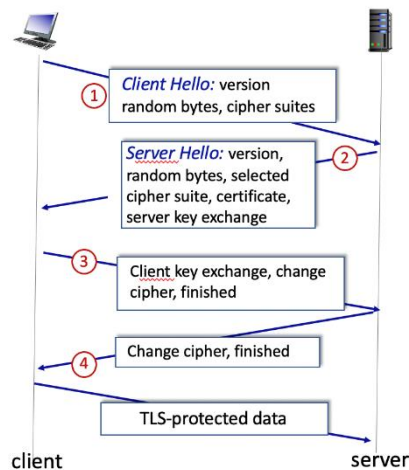


Figure 2: TLS handshaking

3. The TLS Handshake: *Client Hello* message

We learned that, as shown in Figure 2, the client-server TLS handshake begins with the client sending a TLS *Client Hello* message.

3. What version of TLS is your client running, as declared in the *Client Hello* message?
4. How many cipher suites are supported by your client, as declared in the *Client Hello* message? A cipher suite is a set of related cryptographic algorithms that determine how session keys will be derived, and how data will be encrypted and be digitally signed via a HMAC algorithm.
5. Your client generates and sends a string of "random bytes" to the server in the *Client Hello* message. What are the first two hexadecimal digits in the random bytes field of the *Client Hello* message? Enter the two hexadecimal digits (without spaces between the hex digits and without any leading '0x' , using lowercase letters where

¹⁹ For the author's class, when answering the following questions with hand-in assignments, students sometimes need to print out specific packets (see the introductory Wireshark lab for an explanation of how to do this) and indicate where in the packet they've found the information that answers a question. They do this by marking paper copies with a pen or annotating electronic copies with text in a colored font. There are also learning management system (LMS) modules for teachers that allow students to answer these questions online and have answers auto-graded for these Wireshark labs at http://gaia.cs.umass.edu/kurose_ross/lms.htm

needed). *Hint*: be careful to fully dig into the `Random` field to find the `Random Bytes` subfield (do not consider the `GMT UNIX Time` subfield of `Random`).

6. What is the purpose(s) of the “random bytes” field in the *Client Hello* message?
Note: you’ll have to do some searching and reading to get the answer to this question; see section 8.6 and in [RFC 5246](#) (section 8.1 in RFC 5246 in particular).

3. The TLS Handshake: *Server Hello* message

Next, let’s take a look at the second step of the TLS handshake, the *TLS Server Hello* message, which is sent in response to the earlier *TLS Client Hello* message.

7. Which cipher suite has been chosen by the server from among those offered in the earlier *Client Hello* message?
8. Does the *Server Hello* message contain random bytes, similar to how the *Client Hello* message contained random bytes? And if so, what is/are their purpose(s)?

In response to the initial *Client Hello* message, the server also sends back two important additional pieces of information (that may be contained in a different packet than the initial part of the *Server Hello* message). The first important piece of information returned is a certificate issued by a trusted certification authority (CA; see section 8.3 in the text) that binds the public key (and other information) of the web server to that web server’s identity. Your client may use the server’s public key for a number of different purposes, including deriving the symmetric keys to encrypt data being sent over this HTTPS/TLS/TCP session and for generating HMAC digital signatures. You can, of course, look at the server’s certificate in Wireshark. You can also view the server’s certificate in your web browser after the `www.cs.umass.edu` server has responded to your request (and the certificate formatted a lot prettier too) – [here’s how](#).

The second piece of additional information returned from the server are parameters needed for the symmetric encryption of the data (HTTP messages in this case) being exchanged.

Let’s dig a bit deeper into the public key certificate.

9. What is the packet number in your trace for the TLS message part that contains the public key certificate for the `www.cics.umass.edu` server (actually the `www.cs.umass.edu` server)?
10. A server may return more than one certificate. If more than one certificate is returned, are all of these certificates for `www.cs.umass.edu`? If not all are for `www.cs.umass.edu`, then who *are* these other certificates for? You can determine who the certificate is for by checking the `id-at-commonName` field in the returned certificate.
11. What is the name of the certification authority that issued the certificate for `id-at-commonName=www.cs.umass.edu`?
12. What digital signature algorithm is used by the CA to sign this certificate? Hint: this information can be found in `signature` subfield of the `SignedCertificate` field of the certificate for `www.cs.umass.edu`.
13. Look in your trace to find messages between the client and a CA to get the CA’s public key information, so that the client can verify that the CA-signed certificate sent by the server is indeed valid and has not been forged or altered. Do you see such

message in your trace? If so, what is the number in the trace of the first packet sent from your client to the CA? If not, explain why the client did not contact the CA.